Use a Lock Hierarchy

Although this is really a particular case of defining lock ordering, a lock hierarchy can provide a means of checking that the convention is adhered to at runtime. The idea is that you divide your application into layers and identify all the mutexes that may be locked in any given layer. When code tries to lock a mutex, it isn't permitted to lock that mutex if it already holds a lock from a lower layer. You can check this at runtime by assigning layer numbers to each mutex and keeping a record of which mutexes are locked by each thread. The following listing shows an example of two threads using a hierarchical mutex.

Listing 3.7. Using a lock hierarchy to prevent deadlock

```
hierarchical_mutex high_level_mutex(10000);     <-1
hierarchical_mutex low_level_mutex(5000);           <-2

int do_low_level_stuff();

int low_level_func()
{
    std::lock_guard<hierarchical_mutex> lk(low_level_mutex);    <-3
    return do_low_level_stuff();
}

void high_level_stuff(int some_param);

void high_level_func()
{
    std::lock_guard<hierarchical_mutex> lk(high_level_mutex);      <-4
    high_level_stuff(low_level_func());       <-5
}
```